Day 17 — cross-validation and hyperparameter tuning

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 3, Day 3 — paired with day17_cross_validation.py

1. One split was never enough

Day 16's comparison table showed logistic regression beating random forest on one particular 80/20 split. Change the random_state and that ranking can flip. A single test-set score is one sample of how a model performs — cross-validation gives several samples instead of one, and the script proves this before explaining the fix.

```
for seed in range(5):
    X_a, X_b, y_a, y_b = train_test_split(
        X_train, y_train, test_size=0.25, random_state=seed, stratify=y_train
    )
    model = LogisticRegression(max_iter=1000)
    model.fit(X_a, y_a)
    acc = accuracy_score(y_b, model.predict(X_b))
```

In a real run, the exact same model and code scored anywhere from 0.700 to 0.733 across 5 different random splits — a 0.033 spread from nothing but which rows happened to land in which split. That's the instability cross-validation exists to fix.

2. K-Fold cross-validation

Split the training data into K equal chunks ("folds"). Train on K-1 of them, test on the one left out. Repeat K times, rotating which fold is held out. With 5-fold CV, every row is used for testing exactly once and for training four times — no single lucky or unlucky split decides the result.

3. cross_val_score

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X_train, y_train, cv=5)
scores.mean(), scores.std()
```

- Returns an array of 5 accuracy scores, one per fold, not just one number
- The mean is a far more reliable estimate than any single train/test split
- The standard deviation shows how much performance varies across folds — a high std means the model is sensitive to exactly which rows it happens to see

4. Hyperparameters vs learned parameters

A model's parameters (like logistic regression's coefficients) are learned automatically from training data. Hyperparameters — `max_depth`, `n_estimators`, `min_samples_leaf` — are settings chosen before training even starts. Picking good ones is itself a search problem, not a guess.

5. GridSearchCV

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [4, 8, None],
    'min_samples_leaf': [1, 5],
}
search = GridSearchCV(forest, param_grid, cv=5, n_jobs=-1)
search.fit(X_train, y_train)
```

- Tries every combination in the grid — 2 × 3 × 2 = 12 combinations here
- Runs 5-fold CV on each combination, not just one split per setting
- `n_jobs=-1` runs combinations in parallel across all available CPU cores
- `search.best_params_` and `search.best_score_` give you the winner

6. Comparing tuned vs default, honestly

In a real run: default random forest CV scored 0.683 ± 0.046, the tuned version found by GridSearchCV scored 0.733 — a genuine +0.050 improvement, not a rounding artifact. But that won't always be true. On a small, fairly clean dataset, tuning sometimes barely moves the needle at all — that's still a legitimate result. The value of this step is the discipline of checking, not assuming a gain exists.

7. Reading results honestly: mean and spread together

Report both the mean and the standard deviation across folds ("0.72 ± 0.04"), never the mean alone. A model at 0.81 ± 0.01 is more dependable than one at 0.81 ± 0.09, even though their averages match exactly — the second one is far more likely to disappoint on a new batch of data.

The script also enforces one more discipline: the held-out test set from step 0 is never touched until the very last few lines, after every tuning decision has already been made using cross-validation on the training data alone. Peeking at test performance earlier and adjusting based on it would quietly turn the test set into a second training set.

8. Deliverable and push

```
git add day17_cross_validation.py
git commit -m "Day 17: Cross-validation and hyperparameter tuning - GridSearchCV, fold
git push
```